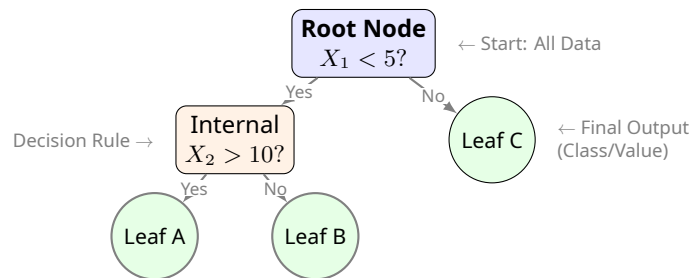


Decision Tree: Regression & Classification

From If-Else Rules to Machine Learning Models

1 Core Idea

What is a Decision Tree? A supervised learning algorithm that makes predictions by learning simple decision rules inferred from data features. It works like a flowchart of **if-else** questions.



Key Terms

- **Root Node:** The starting point; represents the entire dataset.
- **Internal Node:** A decision point involving a feature (e.g., Age > 18).
- **Split:** Dividing a node into sub-nodes.
- **Leaf Node:** Terminal node with the final outcome (Class/Value).

2 Types of Trees

Classification Tree

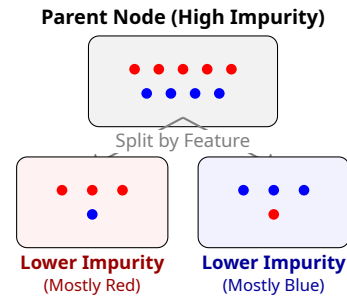
- **Output:** Categorical (Class label).
- **Ex:** Spam or Not Spam?

Regression Tree

- **Output:** Continuous (Number).
- **Ex:** House Price Prediction.

3 Splitting Criteria

The "Best Split" minimizes impurity (Chaos).



Classification Metrics

- **Gini Impurity** (Default): Measures prob. of misclassifying a random element. Range [0, 0.5]. Faster.

$$G = 1 - \sum p_i^2$$

- **Entropy:** Measures information/disorder. Range [0, 1]. Slower (log calc).

$$E = - \sum p_i \log_2(p_i)$$

- **Info Gain:** Entropy(Parent) - Weighted Avg Entropy(Children).

Regression Metrics

- **MSE** (Mean Squared Error): Variance reduction.
- **MAE** (Mean Absolute Error): Robust to outliers.

4 Tree Building Process

Recursive Partitioning (Greedy Algorithm)

1. **Select:** Iterate through every feature.
2. **Find:** Calculate the metric (Gini/MSE) for every possible split point.
3. **Split:** Choose the split that decreases impurity the most.
4. **Repeat:** Apply logic recursively to child nodes.
5. **Stop:** When a stopping condition is met.

5 Stopping Conditions

When does the tree stop growing?

- **max_depth:** Maximum height reached.
- **min_samples_split:** Node has too few samples to split further.
- **min_samples_leaf:** Creating a child would result in too few samples.
- **Pure Node:** All samples belong to one class (Impurity = 0).

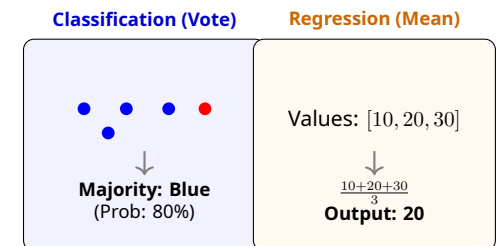
6 Leaf Node Prediction

Classification

- **Prediction:** The majority class in the leaf.
- **Probability:** % of samples of that class in the leaf.

Regression

- **Prediction:** The **Mean** (average) of target values in the leaf.

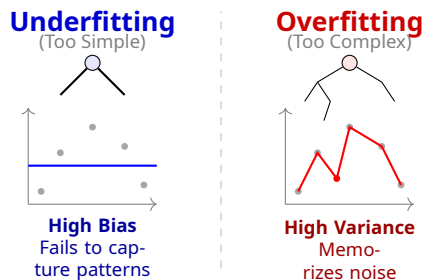


7 Key Hyperparameters

Control complexity to prevent overfitting.

```
model = DecisionTreeClassifier(
    max_depth=5,          # Max levels
    min_samples_split=10, # Min to split
    min_samples_leaf=5,   # Min in leaf
    max_features='sqrt',  # Feats to consider
    criterion='gini'       # 'entropy'/'log_loss'
)
```

8 Over/Underfitting



Overfitting (High Variance)

- Tree is too deep/complex.
- Memorizes noise in training data.
- Perfect train score, poor test score.

Underfitting (High Bias)

- Tree is too shallow (stumps).
- Fails to capture patterns.
- Poor train and test score.

Bias-Variance Tradeoff: We want a balanced tree (neither too deep nor too shallow).

9 Pruning

Trimming the tree to improve generalization.

Pre-Pruning (Early Stopping)

- Stop growing *during* training.
- Uses `max_depth`, `min_samples_leaf`.
- *Pros:* Fast. *Cons:* Might stop too early.

Post-Pruning (Cost Complexity)

- Grow full tree, then cut branches that add little predictive power.
- Uses `ccp_alpha` parameter.

10 Implementation (Code)

Classification (Full Workflow)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# 1. Load & Split
df = pd.read_csv('data.csv')
X = df.drop('target', axis=1) # Features
y = df['target']             # Labels

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 2. Train
clf = DecisionTreeClassifier(max_depth=3)
clf.fit(X_train, y_train)

# 3. Predict & Evaluate
y_pred = clf.predict(X_test)
print(f"Acc: {accuracy_score(y_test, y_pred):.2f}")
```

Regression (Full Workflow)

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error

# 1. Create Dummy Data
X = np.random.rand(100, 5) # 5 Features
y = np.random.rand(100)    # Continuous Target

# 2. Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Train
reg = DecisionTreeRegressor(max_depth=5)
reg.fit(X_train, y_train)

# 4. Predict & Evaluate
y_pred = reg.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print(f"MSE: {mse:.4f}")
```

11 Advantages & Limitations

Pros

- **Interpretability:** Easy to visualize and explain ("White Box").
- **No Scaling:** Works without normalization/standardization.
- **Non-Linear:** Handles complex non-linear relationships well.

Cons

- **High Variance:** Small data change → Completely different tree.
- **Overfitting:** Easy to build overly complex trees.
- **Instability:** Sensitive to noise.

12 When to Use

- When **explainability** is critical (Banking, Medical).
- As a **baseline** model.
- For small to medium structured datasets.
- When features are mixed (numerical & categorical).

13 Quick Comparison

- **vs Linear Reg:** Trees handle non-linear data better; Linear is better for trends.
- **vs Logistic Reg:** Trees handle outliers/missing data better; Logistic gives smoother probabilities.
- **vs Random Forest:** RF is an ensemble of trees (reduces variance, loses interpretability).

14 Real-World Examples

Classification

- **Loan Approval:** Risk level (Low/High).
- **Medical Diagnosis:** Disease present? (Yes/No).
- **Churn Prediction:** Will customer leave?

Regression

- **Pricing:** Estimating used car prices.
- **Demand:** Predicting daily sales volume.

15 Next Steps

Why upgrade? Single trees are weak learners. To improve performance:

- **Random Forest:** Bags multiple trees to reduce variance (Parallel).
- **Gradient Boosting (XGBoost):** Trains trees sequentially to fix errors of previous ones (Sequential).